

Teste de Software

Prof. Iaçanã Ianiski Weber



Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

Software Testing

✓ <https://www.cs.virginia.edu/~up3f/cs3250/syllabus.html>

Márcio Eduardo Delamaro, José Carlos Maldonado e Mario Jino. **Introdução ao Teste de Software**. 2007.

✓ <https://www.amazon.com.br/Introdu%C3%A7%C3%A3o-Software-M%C3%A1rcio-Eduardo-Delamaro/dp/8535226346/>

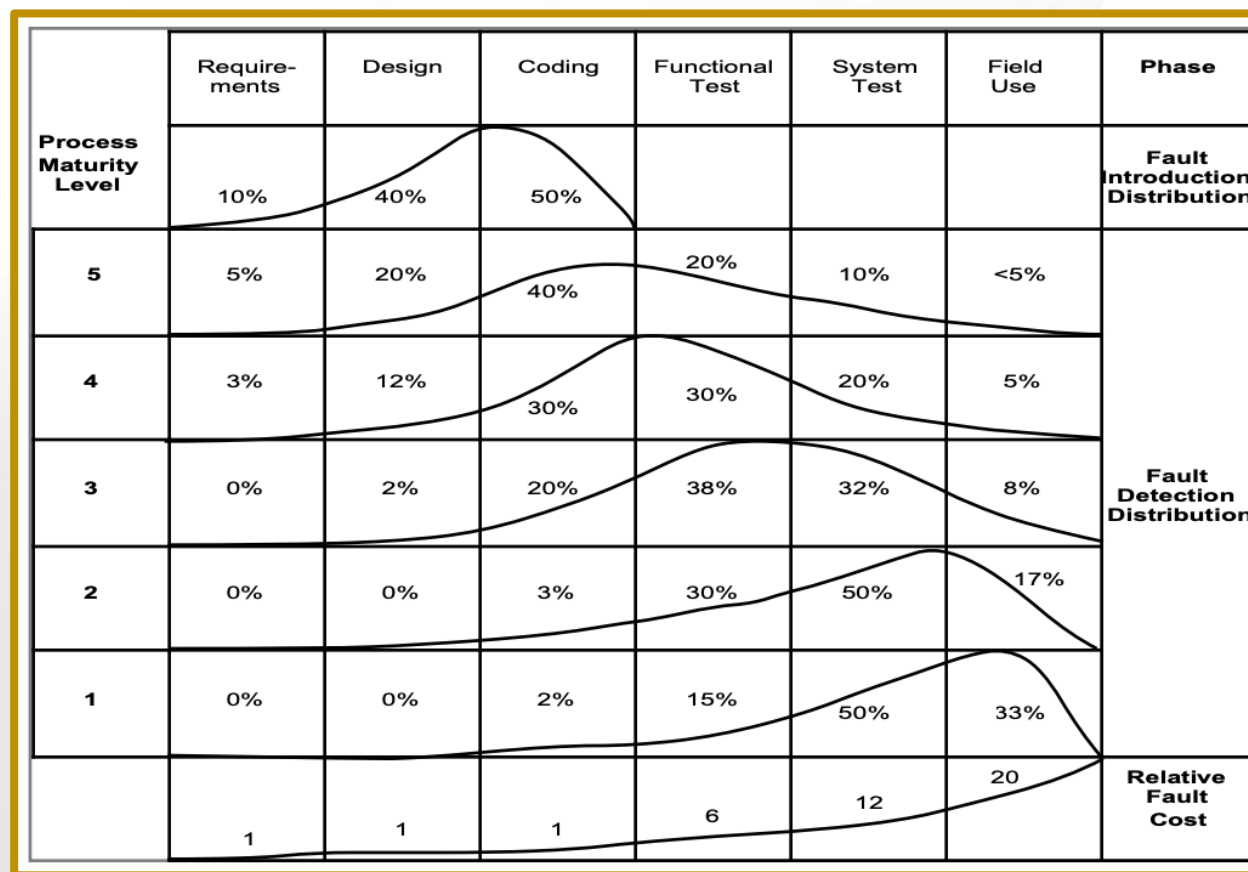
“A qualidade do software está se tornando cada vez mais essencial para todas as empresas e o conhecimento de teste de software está se tornando necessário para todos os engenheiros de software”*

- ✓ *Safety critical*
- ✓ *Real-time software*
- ✓ *Embedded software*
- ✓ *Enterprise applications*
- ✓ *Security*
- ✓ *Web*
- ✓ *Mobile*

*Fonte: Paul Ammann and Jeff Offutt. **Introduction to Software Testing**. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

- ✓ Mudança nas distribuições de falhas conforme aumenta a maturidade do processo

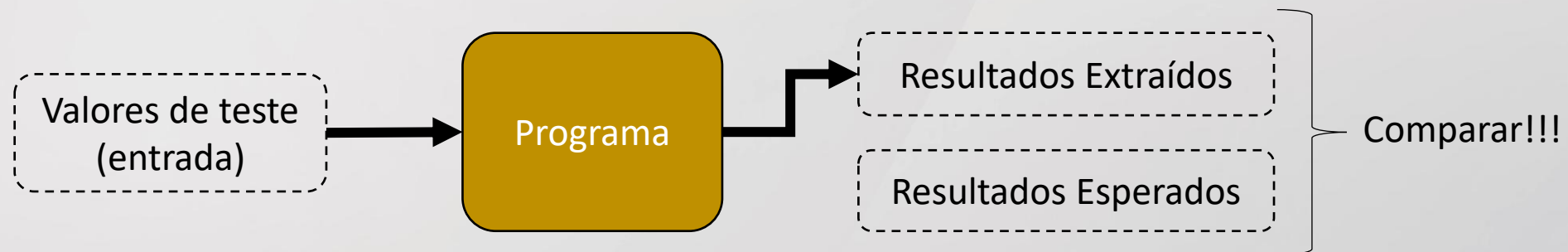


Fonte: Robert E. Park, Wolfhart B. Goethert and William A. Florac. **Goal-Driven Software Measurement — A Guidebook**. 1996.

✓ https://resources.sei.cmu.edu/asset_files/Handbook/1996_002_001_16436.pdf

- ✓ **Teste:** processo de encontrar entradas para verificar um software
 - Um caso de teste consiste de estímulos de entrada e saídas esperadas
 - Teste somente mostra a presença de falhas, não a ausência
 - Nem todos os estímulos vão causar uma falha !!!
- ✓ **Debug:** processo de encontrar um defeito causador de uma falha

“O verdadeiro objetivo do testador não é realizar os testes, mas se preocupar com o desenvolvimento dos casos de teste!”



*Fonte: Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

- ✓ Se teste é o processo de **encontrar entradas** para verificar um software, como lidar com a complexidade ?!?!?
- ✓ Mesmo os problemas mais simples possuem um espaço de estados gigante
 - Considere a função para calcular a média entre 3 inteiros
 - *int average(int a, int b, int c)*
 - Cada variável de 32-bits pode ter 4 bilhões de valores possíveis
 - Combinação de $C(3, 4\text{Bilhões}) > 80 \text{ octilhões}$ de testes possíveis !!!

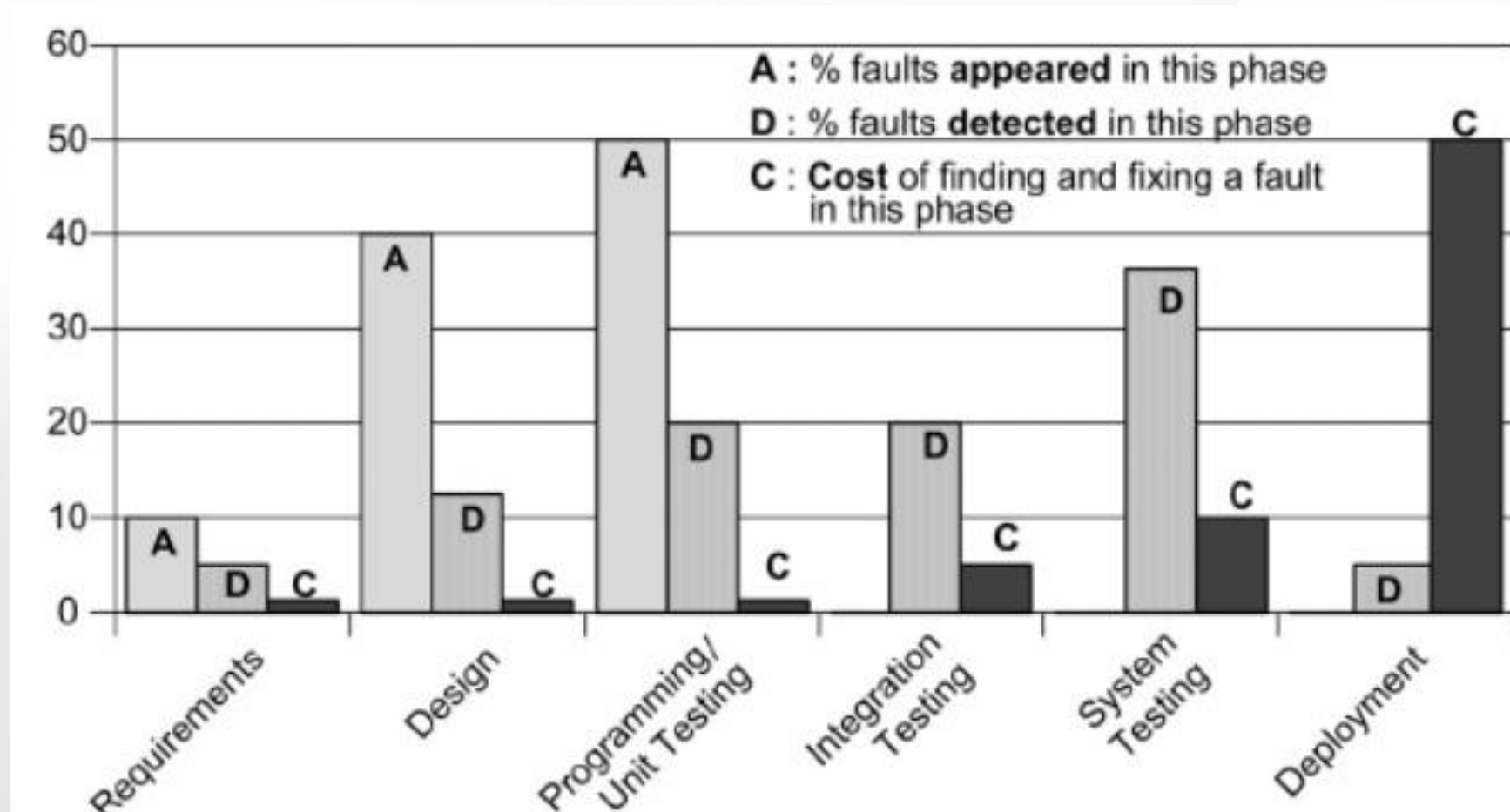
NOME	NÚMERO
Um	1
Dez	10
Cem	100
Mil	1.000
Milhão	1.000.000
Bilhão	1.000.000.000
Trilhão	1.000.000.000.000
Quatrilhão	1.000.000.000.000.000
Quintilhão	1.000.000.000.000.000.000
Sextilhão	1.000.000.000.000.000.000.000
Septilhão	1.000.000.000.000.000.000.000.000
Octilhão	1.000.000.000.000.000.000.000.000.000
Nonilhão	1.000.000.000.000.000.000.000.000.000.000
Decilhão	1.000.000.000.000.000.000.000.000.000.000.000

Considere o teste da função x^y

- ✓ O domínio de entrada é formado pelo par (x,y)
- ✓ Cardinalidade é $2^n * 2^n$
- ✓ Se n for 32 bits, isso representa $2^{64} = 18446744073709551616$
- ✓ Se cada caso de teste for executado em 1 ms, vai levar **5.849.424 séculos** para executar todos os testes!

Fonte: Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>



Fonte: Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. 2nd edition. 2017.

✓ <https://cs.gmu.edu/~offutt/softwaretest/>

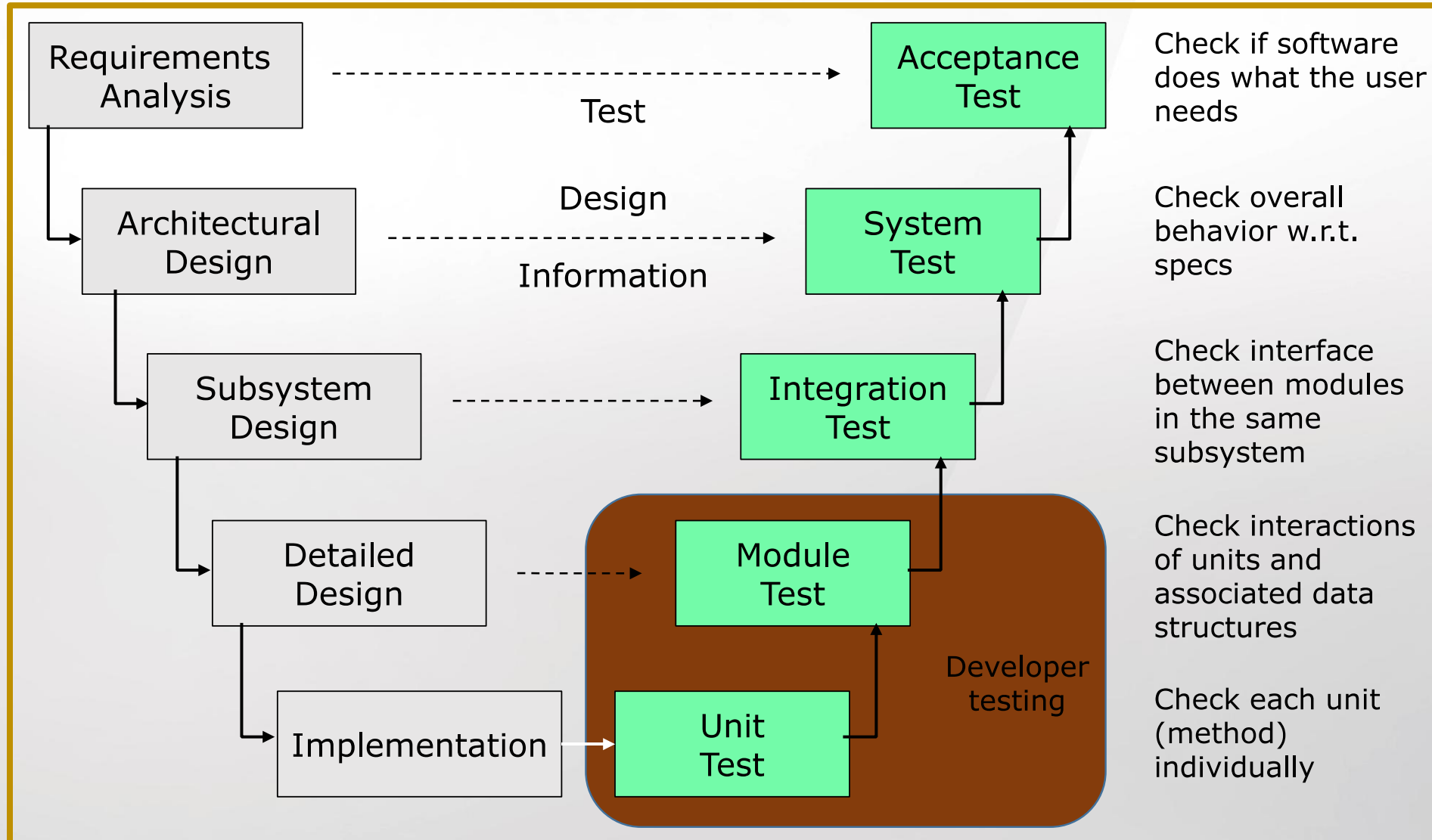
O objetivo é **eliminar os defeitos mais cedo possível**

- ✓ Melhorar a qualidade
- ✓ Reduzir custos
- ✓ Preservar satisfação do usuário

- ✓ **Teste Estático:** testa sem executar o software
 - Realiza inspeção de software e análise (formal)
 - Exemplos: verificação formal, lint checkers
 - Vamos ver algumas ferramentas, mas não é o foco da disciplina

- ✓ **Teste dinâmico:** teste executando o programa com entradas reais
 - É o **foco** da disciplina

- ✓ **Validação**: realizado no final do processo de desenvolvimento **por usuários** para verificar se o software atende as necessidades
- ✓ **Verificação**: realizado **por desenvolvedores** durante o desenvolvimento do software para verificar se atende os requisitos



Se considerarmos os últimos 30 anos da computação...

O teste de software se tornou mais **fácil** e **difícil** do que nunca!

Difícil pois...

- Vasta gama de linguagens de programação
- Muitas plataformas de hardware
- Diversos Sistemas Operacionais
- Onipresença do software (altamente difundido na sociedade)

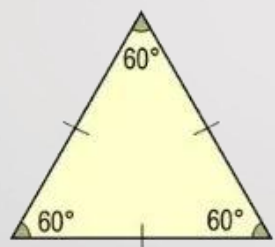
Fácil pois...

- Sistemas Operacionais modernos (serviços e rotinas bem testadas)
 - GUIs, por exemplo, facilmente desenvolvidas a partir de bibliotecas
- Necessidade reduzida de testar objetos pré-programados
- Inteligência Artificial para ajudar no desenvolvimento dos testes

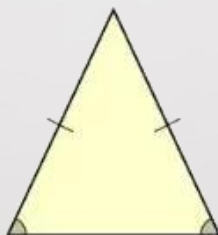
Agora, antes de começarmos... Vamos fazer um teste!

Escreva um conjunto de **casos de teste** – *conjuntos específicos de dados* – para testar adequadamente um programa relativamente simples:

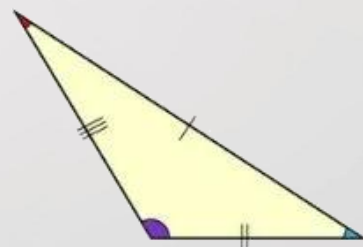
O programa recebe três valores inteiros. Os três valores representam os comprimentos dos lados de um triângulo. O programa exibe uma mensagem informando se o triângulo é escaleno, isósceles ou equilátero.



Equilátero



Isósceles



Escaleno

Lembre-se de que um triângulo **escaleno** é aquele em que não há dois lados iguais, enquanto um triângulo **isósceles** tem dois lados iguais e um triângulo **equilátero** tem três lados de comprimento igual. Além disso, os ângulos opostos aos lados iguais num triângulo isósceles também são iguais, e todos os ângulos num triângulo equilátero são iguais.

Vamos avaliar as respostas (1 ponto para cada “sim”):

1) Você tem um caso de teste que representa um triângulo escaleno válido?

(Observe que casos de teste como 1, 2, 3 e 2, 5, 10 não garantem uma resposta sim porque um triângulo com essas dimensões não é válido.)

2) Você tem um caso de teste que representa um triângulo equilátero válido?

3) Você tem um caso de teste que representa um triângulo isósceles válido?

(Observe que um caso de teste representando 2, 2, 4 não contaria porque não é um triângulo válido.)

4) Você tem pelo menos três casos de teste que representam triângulos isósceles válidos, de modo que você tentou todas as três permutações de dois lados iguais?

(Por exemplo: 3, 3, 4; 3, 4, 3; 4, 3, 3.)

5) Você tem um caso de teste em que um lado tem valor zero?

6) Você tem um caso de teste em que um lado tem um valor negativo?

7) Você tem um caso de teste com três inteiros maiores que zero, tal que a soma de dois números é igual ao terceiro?

(Ou seja, se o programa dissesse que 1, 2, 3 representa um triângulo escaleno, conteria um bug.)

Vamos avaliar as respostas (1 ponto para cada “sim”):

8) Você tem um caso de teste com três números inteiros maiores que zero, de modo que a soma de dois números seja menor que o terceiro?

(Como 1, 2, 4 ou 12, 15, 30)

9) Você tem pelo menos três casos de teste na categoria 9, de modo que tenha tentado todas as três permutações?

(Por exemplo, 1, 2, 4; 1, 4, 2; e 4, 1, 2)

10) Você tem um caso de teste em que todos os lados são zero (0, 0, 0)?

11) Você tem pelo menos um caso de teste especificando valores não inteiros?

12) Você tem pelo menos um caso de teste especificando o número errado de valores?

13) Para cada caso de teste você especificou a saída esperada do programa além dos valores de entrada?

É claro que um conjunto de casos de teste que satisfaça essas condições não garante que você encontrará todos os erros possíveis!

Entretanto, as questões de 1 a 13 representam erros que realmente ocorreram em diferentes versões deste programa, um teste adequado deste programa deve expor pelo menos esses erros.

E então, qual foi sua pontuação???

Programadores profissionais altamente qualificados pontuam, em média, apenas **7,8/14**.

[The Art of Software Testing]

- ✓ O objetivo do teste que fizemos é ilustrar que testar um programa trivial como este já não é uma tarefa fácil.
- ✓ Dado que isto é verdade, imaginemos a dificuldade de testar um sistema de controle de tráfego aéreo com 100.000 requisitos, um compilador ou mesmo um programa de folha de pagamento.
- ✓ Ao trabalhar com este exemplo, pode parecer assustador testar minuciosamente um programa complexo do mundo real.

Embora a tarefa possa ser assustadora, o teste adequado de programas é uma parte **muito necessária** — e **alcançável** — do desenvolvimento de software, como veremos nesta disciplina.

- ✓ Consiste em chamar uma função ou método de uma classe (*unit*) e executá-la com um conjunto de parâmetros conhecidos.
- ✓ O resultado é então comparado com o resultado esperado.

- ✓ Se essa expressão *assert* for avaliada como 0, isso causará uma falha (*assertion failure*) que encerrará o programa.

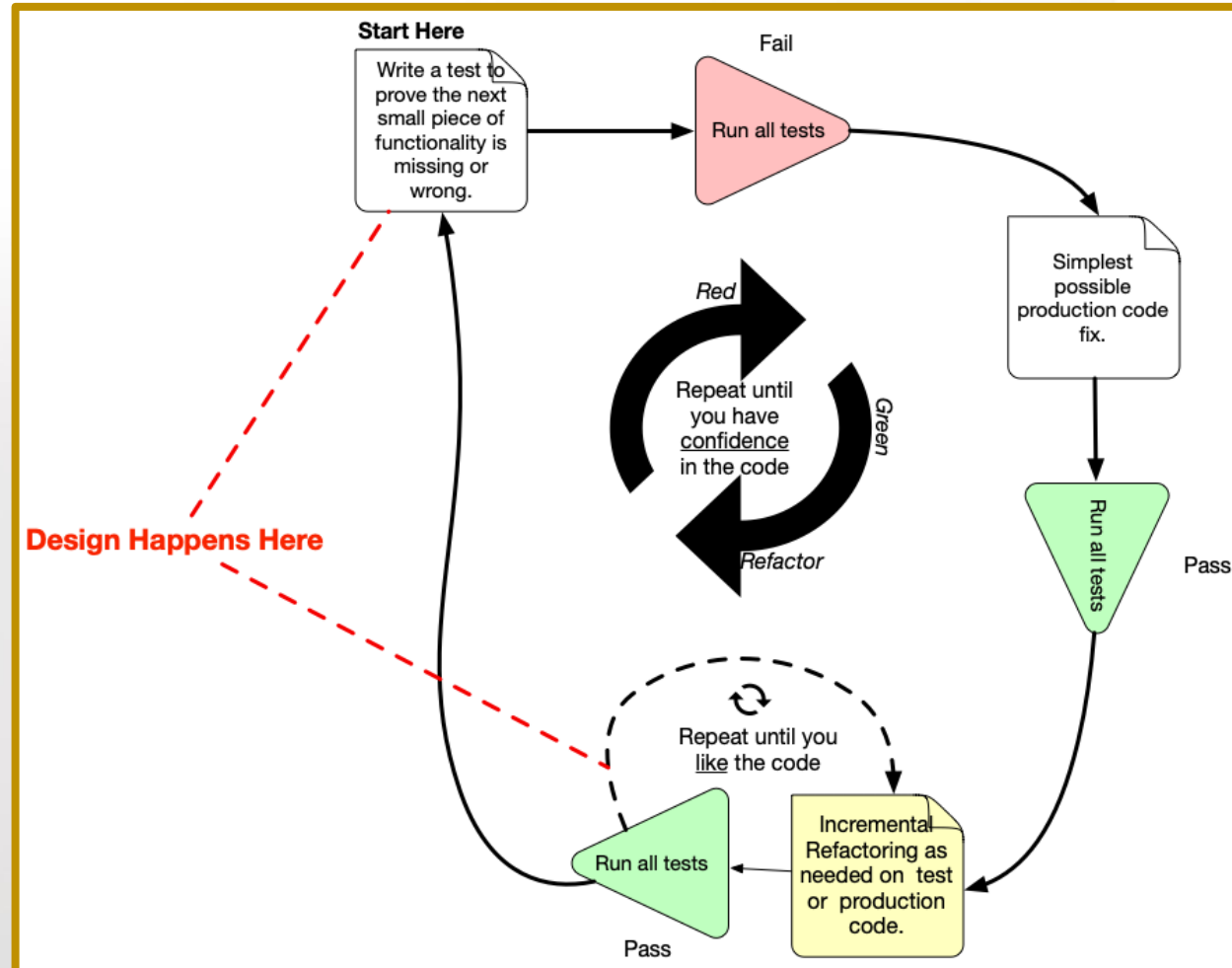
```
int foo(int a, int b) {  
    return a-b-1;  
}  
  
void test_foo() {  
    int ret = foo(1, 2);  
    assert(ret == -2, "wrong value returned")  
}
```

- ✓ Segundo o livro, os testes devem ser **FIRST**:
 - **Fast**: Os testes devem ser rápidos, rápidos o suficiente para que os desenvolvedores os executem em cada pequena alteração, sem esperar a fim de não interromperem o fluxo do projeto.
 - **Isolated**: Os testes devem ser isolados. Um teste não deve impactar o próximo teste. Os testes também isolam as falhas.
 - **Repeatable**: Os testes são repetíveis, repetível significa automatizado. Os testes são executados em um loop sempre dando o mesmo resultado.
 - **Self-verifying**: Os testes verificam seus resultados, relatando um simples "OK" quando são aprovados, enquanto fornecem detalhes concisos quando eles falham.
 - **Timely**: Os testes são feitos a tempo. Os programadores escrevem os testes em uma etapa diferente, isto é, normalmente ele é escrito um pouco antes do código de produção, evitando bugs.

- ✓ É uma técnica para desenvolvimento de software incremental
- ✓ Filosofia:
 1. Desenvolva o teste antes
 2. Execute o teste, gerando falhas
 3. Só depois desenvolva o código até que os testes passem
 4. Repita o processo para novas funcionalidades
- ✓ O teste representa o comportamento desejado do código
- ✓ A cada mudança no código, execute novamente os testes

- A chave é começar pequeno. Testes simples.
- Aumente seu código lentamente.
- Só torne seu código mais complexo se os testes exigirem isso.
- Pense em uma boa sequência de testes

✓ Máquina de Estados do TDD:



- ✓ É um software que permite o programador expressar o comportamento dos testes
- ✓ Deve possuir:
 - Uma linguagem para descrever os testes e valores esperados
 - Mecanismo para executar os testes
 - Relatórios dos teste

- ✓ Framework para Teste de Unidade em C
 - <https://github.com/ThrowTheSwitch/Unity>
- ✓ Motivação:
 - Pouca dependência de bibliotecas
 - Pode ser utilizado para testar Sistemas Embarcados - **Baremetal**, isto é (*sem OS*)
- ✓ Livro: James W. Grenning. ***Test-Driven Development for Embedded C***. 2011.
 - Capítulo 3
 - Fonte: https://pragprog.com/titles/jgade/source_code

- ✓ Abra o terminal e vá para o diretório onde você vai guardar o projeto
- ✓ Execute:
 - git clone <https://github.com/iacanaw/Unity.git>
 - cd Unity/examples/foo
 - make all

✓ **sprintf**

- Compõe uma string com o mesmo texto que seria impresso se fosse usado em printf, mas em vez de ser impresso, o conteúdo é armazenado como uma string C no buffer apontado pelo 1º argumento.
- Retorna o número de bytes que foi copiado para essa string!

- ✓ TEST (<group-of-test>, <test-name>)
- ✓ TEST_ASSERT_EQUAL – compara 2 inteiros
- ✓ TEST_ASSERT_EQUAL_STRING – compara string C

```
TEST(sprintf, NoFormatOperations)
{
    char output[5] = "";
    TEST_ASSERT_EQUAL(3, sprintf(output, "hey"));
    TEST_ASSERT_EQUAL_STRING("hey", output);
}
```

- ✓ E se sprintf usar mais de 12 bytes ?

```
TEST(sprintf, InsertString)
{
    char output[20] = "";

    TEST_ASSERT_EQUAL(12, sprintf(output, "Hello %s\n", "World"));
    TEST_ASSERT_EQUAL_STRING("Hello World\n", output);
}
```

- ✓ Esse teste verifica bytes após a string

```
TEST(sprintf, InsertString)
{
    char output[20];
    memset(output, 0xaa, sizeof output);

    TEST_ASSERT_EQUAL(12, sprintf(output, "Hello %s\n", "World"));
    TEST_ASSERT_EQUAL_STRING("Hello World\n", output);
    TEST_ASSERT_BYTES_EQUAL(0xaa, output[13]);
}
```

Copio 0xaa em todas as posições do output

Faço a verificação se as outras posições não foram alteradas

- ✓ Evitando duplicação de código entre os testes

```
TEST_GROUP(sprintf);

static char output[100];
static const char * expected;

TEST_SETUP(sprintf)
{
    memset(output, 0xaa, sizeof output);
    expected = "";
}

TEST_TEAR_DOWN(sprintf)
{
}
```


- ✓ Evitando duplicação de código entre os testes

```
static void expect(const char * s)
{
    expected = s;
}

static void given(int charsWritten)
{
    TEST_ASSERT_EQUAL(strlen(expected), charsWritten);
    TEST_ASSERT_EQUAL_STRING(expected, output);
    TEST_ASSERT_BYTES_EQUAL(0xaa, output[strlen(expected) + 1]);
}
```

- ✓ Evitando duplicação de código entre os testes

```
TEST(sprintf, NoFormatOperations)
{
    expect("hey");
    given(sprintf(output, "hey"));
}

TEST(sprintf, InsertString)
{
    expect("Hello World\n");
    given(sprintf(output, "Hello %s\n", "World"));
}
```

✓ Agrupando testes

```
#include "unity_fixture.h"

TEST_GROUP_RUNNER(sprintf)
{
    RUN_TEST_CASE(sprintf, NoFormatOperations);
    RUN_TEST_CASE(sprintf, InsertString);
}
```

✓ Função *main* do teste

```
#include "unity_fixture.h"

static void RunAllTests(void)
{
    RUN_TEST_GROUP(sprintf);
}

int main(int argc, char * argv[])
{
    return UnityMain(argc, argv, RunAllTests);
}
```

✓ *Unity output sem erros*

```
make
compiling SprintfTest.c
Linking BookCode_Unity_tests
Running BookCode_Unity_tests
..
-----
2 Tests 0 Failures 0 Ignored
OK
```


✓ *Unity output com erros*

```
TEST(sprintf, NoFormatOperations)
{
    char output[5];

    TEST_ASSERT_EQUAL(4, sprintf(output, "hey"));
    TEST_ASSERT_EQUAL_STRING("hey", output);
}
```



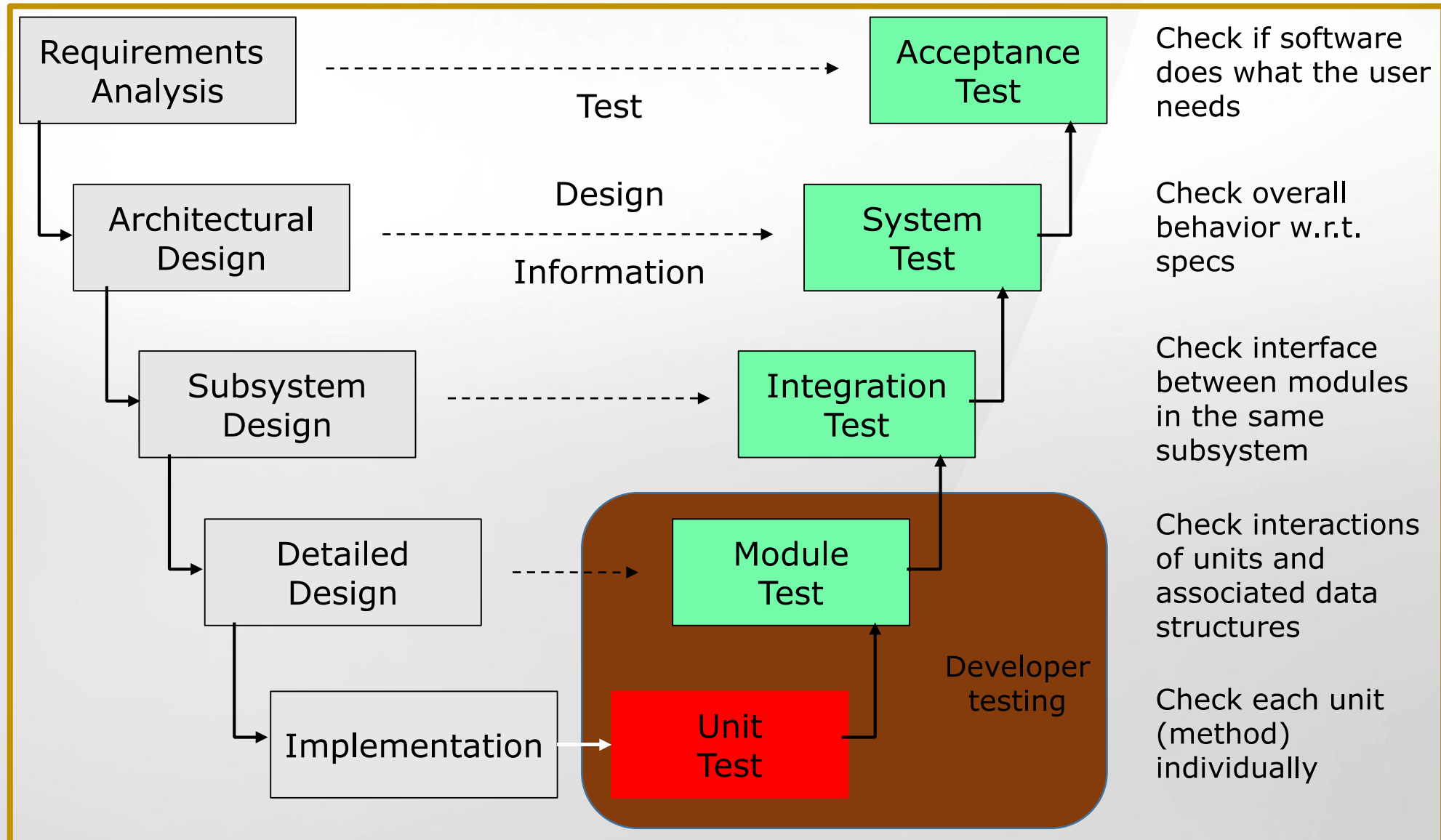
```
make
compiling SprintfTest.c
Linking BookCode_Unity_tests
Running BookCode_Unity_tests
..
TEST(sprintf, NoFormatOperations)
    stdio/SprintfTest.c:75: FAIL
    Expected 4 Was 3
-----
2 Tests 1 Failures 0 Ignored
FAIL
```

ATIVIDADE

1. Fazer o clone do repositório **Unity**
 - Referência: <https://github.com/iacanaw/Unity.git>
2. Replicar a pasta foo e adaptar para a *função dos triângulos*.
 - **cd Unity/examples**
 - **cp -fr foo triangulos**
3. Implementar os **testes unitários** para a função *função dos triângulos*
 - O arquivo **UnityAssertionsReference.md** em **Unity/docs** apresenta referências à todas as *assertions* disponíveis.

DELAMARO, M.E.; MALDONADO J.C.; JINO, M.. *Introdução ao Teste de Software*. Elsevier. 2007.

- Capítulo 2



- ✓ Técnicas:
 - Funcional (Capítulo 2)
 - Estrutural (Capítulo 4)
 - Baseada em Erros (Capítulo 5)
- ✓ A diferença está na fonte utilizada para *definir os requisitos de teste!!*
- ✓ Mas o **objetivo** das técnicas é similar:
 - Conduzir o processo de teste de uma forma mais sistemática
 - Escolher casos de teste que representem o domínio

- ✓ O software é considerado uma **caixa preta**
- ✓ No teste funcional são verificadas as funções do sistema *sem se preocupar com os detalhes de implementação*
- ✓ Entradas e saídas são definidas para verificar conformidade com objetivos ou **especificação**
- ✓ O software é avaliado do ponto de vista do usuário
- ✓ O teste funcional envolve *dois passos principais*:
 1. Identificar as funções que o software deve realizar
 2. Criar casos de teste capazes de checar se essas funções estão sendo realizadas pelo software

1. Particionamento de Equivalência
 2. Análise do Valor Limite
 3. Sistemático
 4. Grafo Causa-Efeito
 5. Error-Guessing
- ✓ A qualidade de tais critérios depende fortemente da existência de uma **boa especificação** de requisitos
 - ✓ Especificações ausentes ou mesmo incompletas tornarão difícil a aplicação dos critérios funcionais

Especificação do programa “Cadeia de Caracteres”:

O programa solicita do usuário um inteiro positivo no intervalo entre 1 e 20 e então solicita uma cadeia de caracteres desse comprimento. Após isso, o programa solicita um caractere e retorna a posição na cadeia em que o caractere é encontrado pela primeira vez ou uma mensagem indicando que o caractere não está presente na cadeia. O usuário tem a opção de procurar vários caracteres.

1. PARTICIONAMENTO DE EQUIVALÊNCIA

- ✓ Objetivo: é particionar o domínio de entrada de forma que os casos de testes dos subconjuntos (partições ou subdomínios) seja uma boa representação de todo o domínio
- ✓ Divide o domínio de entrada em **classes de equivalência** que, de acordo com a especificação do programa, são tratadas da mesma maneira.
 - Isto é, o valor dentro de uma classe é equivalente a todos os outros valores dentro da mesma classe
 - Logo, se eu testar um valor de uma classe, eu estarei cobrindo os outros valores dessa mesma classe
- ✓ Assumption: Se um elemento do subconjunto detectar um defeito, qualquer outro também detecta

- ✓ Pode-se observar a especificação procurando termos como:
 - “*intervalo*” e “*conjunto*” ou
 - Palavras similares que indiquem que os dados são processados da mesma forma
- ✓ Representa um conjunto de estados válidos ou inválidos para as condições de entrada

- ✓ Diretrizes para a definição das classes:
 - Se a condição de entrada especifica um **intervalo de valores**, são definidas uma classe válida e duas inválidas
 - Se a condição de entrada especifica uma **quantidade de valores**, são definidas uma classe válida e duas inválidas
 - Se a condição de entrada especifica um **conjunto de valores determinados manipulados de forma diferente**, é definida uma classe válida para cada um desses valores e uma classe inválida com outro valor qualquer
 - Se a condição de **entrada especifica uma situação do tipo “deve ser de tal forma”**, são definidas uma classe válida e uma inválida
- ✓ Definidas as classes, deve-se definir casos de teste que cubram o maior número de classes

- ✓ 1º passo: definir classes de equivalência
- ✓ 2º passo: definir casos de teste de forma a ter o menor número de casos de teste possível
- ✓ Considerando a especificação dada anteriormente, têm-se **quatro entradas**:
 - **T** – tamanho da cadeia de caracteres
 - **CC** – uma cadeia de caracteres
 - **C** – um caractere a ser procurado
 - **O** – a opção por procurar mais caracteres

- ✓ As variáveis **CC** e **C** **não determinam classes de equivalência**, pois os caracteres podem ser quaisquer
- ✓ O tamanho da cadeia (**T**) deve estar no intervalo entre 1 e 20
- ✓ A opção (**O**) do usuário pode ser “*sim*” ou “*não*”
- ✓ No domínio de **saída**, têm-se duas alternativas:
 1. A posição na qual o caractere foi encontrado na cadeia de caracteres
 2. Uma mensagem declarando que o caractere não foi encontrado
- ✓ Uma partição (**SCC**) onde a string **contém o caractere**, e outra que **não contém**

✓ Classes de equivalência da especificação Cadeia de Caracteres

Variável	Classes de equivalência válidas	Classes de equivalência inválidas
T	$1 \leq T \leq 20$	$T < 1 \text{ e } T > 20$
O	Sim, Não	Resposta Inadequada
SCC	Caractere que pertence à cadeia	Caractere que não pertence à cadeia

✓ Casos de testes para o critério **Particionamento de Equivalência**

Variáveis de entrada				Saída Esperada
T	CC	C	O	
34				Entre com um inteiro entre 1 e 20
0				Entre com um inteiro entre 1 e 20
3	abc	c		O caractere c aparece na posição 3 da cadeia
			s	<o programa deve continuar>
		k		O caractere k não pertence à cadeia
			n	<o programa deve encerrar>

✓ **Vantagem:** resulta em casos de teste reduzidos

- OK quando as entradas podem ser facilmente identificadas na especificação do problema

✓ **Desvantagem:**

- Quando a entrada é simples, mas o processamento é complexo
- Grupos de dados de mesma partição podem não ser processados de forma idêntica

2. ANÁLISE DO VALOR LIMITE

- ✓ Assumption: Casos de teste que exploram condições limites têm uma maior probabilidade de encontrar defeitos
- ✓ É usado em conjunto com o particionamento de equivalência
- ✓ Ao invés de escolher valores aleatórios, os dados de teste devem ser selecionados de forma **a explorar os limites de cada classe** de equivalência
 - Casos de teste que correspondem a valores **sobre, acima, ou abaixo** dos valores de limite

- ✓ Se uma classe válida estiver **no intervalo $-1,0$ e $+1,0$** , devem ser definidos os seguintes dados de teste:
 - $-1,0$; $1,0$; $-1,001$ e $+1,001$;
- ✓ Se é especificada uma **quantidade de valores**, por exemplo, de 1 a 255, os dados de teste devem ter:
 - nenhum valor de entrada
 - somente um valor de entrada
 - 255 valores de entrada
 - 256 valores de entrada
- ✓ Repetir as diretrizes para condições de saída

✓ Casos de testes para o critério **Análise do Valor Limite**

Variáveis de entrada				Saída Esperada
T	CC	C	O	
21				Entre com um inteiro entre 1 e 20
0				Entre com um inteiro entre 1 e 20
1	a	a		O caractere a aparece na posição 1 da cadeia
			s	<o programa deve continuar>
		x		O caractere x não pertence à cadeia
			n	<o programa deve encerrar>
20	abcdefghijklmnpqrst	a		O caractere a aparece na posição 1 da cadeia
			s	<o programa deve continuar>
		t		O caractere t aparece na posição 20 da cadeia
			n	<o programa deve encerrar>

➤ Quebra da tabela denota uma segunda execução do programa

3. TESTE FUNCIONAL SISTEMÁTICO

- ✓ Combina os critérios de *Particionamento de Equivalência & Análise do Valor Limite*
- ✓ Este critério requer **ao menos dois casos de teste** de cada partição
 - Minimiza que defeitos coincidentes mascarem falhas
- ✓ Diretrizes, tanto para valores de entrada quanto de saída:
 - Para **valores discretos**: testar todos os valores
 - Para **intervalos de valores**: testar os extremos e um valor no interior
 - Se valor tem 16 bits, os valores -32768 e +32767 devem ser selecionados
 - Números reais: deve-se definir uma margem de erro, selecionar valores bem pequenos e zero
 - Texto: deve explorar comprimentos variáveis, alfabéticos, alfanuméricos, caracteres especiais
- ✓ Valores imediatamente **fora do limite** para verificar se o software os rejeita

✓ Casos de testes para o critério Funcional Sistemático

Variáveis de entrada				Saída Esperada
T	CC	C	O	
a				Entre com um inteiro entre 1 e 20
1.0				Entre com um inteiro entre 1 e 20
1	!	"	n	O caractere " não pertence à cadeia
1	}	~	n	O caractere ~ não pertence à cadeia
20	!"#\$%&()*+ '/01234567	!	s	O caractere ! aparece na posição 1 da cadeia
		"	s	O caractere " aparece na posição 2 da cadeia
		+	s	O caractere + aparece na posição 10 da cadeia
		6	s	O caractere 6 aparece na posição 19 da cadeia
		7	n	O caractere 7 aparece na posição 20 da cadeia
2	ab	B	n	O caractere b aparece na posição 2 da cadeia
3	a2b	2	n	O caractere 2 aparece na posição 2 da cadeia

Sistema de Cálculo da Zona de Conforto Ambiental

Deve ser desenvolvido um software em linguagem C que calcula se um ambiente está na zona de conforto ambiental definida pela combinação de temperatura, umidade e velocidade do ar. A zona de conforto é definida pelas seguintes regras: A temperatura deve estar entre 20°C e 25°C (inclusos); A umidade relativa entre 40% e 60% (inclusos); A velocidade do ar de até 0.1 m/s (incluso).

Se todas as condições forem atendidas, o software deve retornar que o ambiente está na zona de conforto. Caso contrário, deve especificar qual condição não foi atendida.

Protótipo:

```
int confAmbiental(float t, float u, float v)
```

Entradas:

- ‘t’ (float): um número racional representando a temperatura
- ‘u’ (float): um número racional representando a umidade
- ‘v’ (float): um número racional representando a velocidade do ar

Saídas:

int: um código (vide tabela) que indica se o ambiente se encontra na zona de conforto (cod: 7) ou qual parâmetro se encontra desregulado.

Código	Temperatura	Umidade	Velocidade
0			
1			✓
2		✓	
3		✓	✓
4	✓		
5	✓		✓
6	✓	✓	
7	✓	✓	✓

✓ Classes de equivalência da especificação:

Variável	Classes de equivalência válidas	Classes de equivalência inválidas
T	$20.0 \geq T \geq 25.0$	$T > 25.0$ e $T < 20.0$
U	$40.0 \geq U \geq 60.0$	$U > 60.0$ e $U < 40.0$
V	$V \leq 0.1$	$V > 0.1$

✓ Análise do Valor Limite:

Temperatura (T):

- Limite Inferior: Testar com 19.9°C (fora) e 20°C (dentro).
- Limite Superior: Testar com 25°C (dentro) e 25.1°C (fora).

Umidade (U):

- Limite Inferior: Testar com 39.9% (fora) e 40% (dentro).
- Limite Superior: Testar com 60% (dentro) e 60.1% (fora).

Velocidade do Ar (V):

- Limite Único: Testar com 0.1 m/s (dentro) e 0.11 m/s (fora).

- ✓ Por ser baseado nos critérios Particionamento de Equivalência e Análise do Valor Limite, o critério Funcional Sistemático apresenta os mesmos problemas que estes
- ✓ Fornece diretrizes para facilitar a geração de casos de testes e enfatiza mais fortemente a seleção de mais de um caso de teste por partição e/ou limite
 - Maior chance de revelar os defeitos
 - Maior probabilidade de obter uma melhor cobertura de código

- ✓ Uma vantagem do teste funcional é que ele se baseia somente na especificação e não depende da forma que o software foi implementado:
 - Procedimental, orientado a objetos, ou componentes
- ✓ ENTRETANTO:
 - E se a especificação está incompleta?
 - E se os elementos de uma mesma classe de equivalência NÃO se comportam da mesma maneira?
 - Também, esse tipo de teste não assegura que partes do código foram cobertas pelos casos de teste

- ✓ Façam o particionamento de classes de equivalência do problema dos triângulos visto na aula passada.
- ✓ Proponham testes baseados no critério Funcional Sistemático (particionamento de classes + análise do valor limite).
- ✓ Implementem testes no ambiente Unity apresentado na aula passada.
- ✓ **A função fornecida possui algum erro segundo os testes de vocês?**

✓ Entradas Inválidas

- Lado não-positivo: pelo menos um dos lados ≤ 0
- Violação da Desigualdade Triangular:
 - $a + b \leq c$
 - $a + c \leq b$
 - $b + c \leq a$
- Risco de overflow: se a soma de dois lados estoura o tipo antes de comparar.

✓ Entradas Válidas

- Que produz saída “Equilátero”
 - lados positivos onde $a == b == c$
- Que produz saída “Isósceles”
 - lados positivos, desigualdade satisfeita, e exatamente dois lados iguais
- Que produz saída “Escaleno”
 - lados positivos, desigualdade satisfeita, onde $a \neq b \neq c \neq a$